Solución S01 y pendientes de Diversify Delivery

Fecha: 9 de noviembre de 2026. Repositorio: kevinxonium153/DiversifyDelivery. Rama de pruebas: beta.

Resultado

Se corrigió el hallazgo de prioridad alta S01: un repartidor ya no puede consultar por ID pedidos sin asignar que todavía estén Recibido o que estén cerrados; tampoco puede quitarse de pedidos entregados o cancelados. La corrección está integrada sobre beta (base 2d55812) y comprobada. No se ha desplegado en producción.

Actualización: también se corrigió S02 (asignación simultánea); ver `docs/SOLUCION-SEGURIDAD-S02.md`. También se corrigió S03 (límite de pendientes); ver `docs/SOLUCION-SEGURIDAD-S03.md`. Los otros cinco hallazgos de seguridad (S04–S08) siguen pendientes. El inventario completo de pendientes de seguridad, operación, producto y decisiones está en notas/Pendientes.md; el PDF `docs/Solucion_S01_y_todos_los_pendientes.pdf` incorpora ese inventario, incluyendo los avances existentes en beta de PixelPay y recuperación de contraseña.

Causa y cambio técnico

Archivo: src/Diversify.Delivery/Modules/Orders.cs, método MapDriver.

La consulta GET /api/driver/orders/{id} aceptaba cualquier pedido con DriverId nulo, aunque el listado de pedidos disponibles solo admitía Preparando o Listo. Conocer o adivinar el ID permitía consultar teléfono, dirección, instrucciones y coordenadas cuando existieran. Ahora la consulta exige que el pedido pertenezca al repartidor autenticado o que esté sin asignar y en Preparando/Listo. Los accesos no permitidos responden 404.

Regla de consulta implementada:

```
x.Id == id &&
(x.DriverId == me.UserId() || (x.DriverId == null &&
    (x.Status == OrderStatus.Preparando || x.Status == OrderStatus.Listo)))
```

La acción POST /api/driver/orders/{id}/release solo prohibía EnCamino. Ahora permite únicamente Preparando y Listo, siempre que quien la solicita sea el repartidor asignado. Los estados Recibido, EnCamino, Entregado y Cancelado se rechazan con 400. Otro repartidor recibe 404. La lectura, validación, actualización y evento de liberación se realizan en una transacción SQLite; un rechazo no persiste cambios.

Validación añadida, además de conservar el mensaje existente para EnCamino:

```
AppException.ThrowIf(o.Status is not (OrderStatus.Preparando or OrderStatus.Listo),
    "Solo puedes soltar pedidos que se estén preparando o estén listos. Los pedidos cerrados
conservan su repartidor.");
```

La transacción se inicia antes de cargar el pedido y se confirma después de SaveChangesAsync. No se cambiaron los datos históricos ni el esquema de base de datos. La carrera al tomar pedidos se corrigió posteriormente en S02.

Comportamiento esperado

- Pedido Recibido sin repartidor: consulta por repartidor devuelve 404.

- Pedido Preparando o Listo sin repartidor: consulta permitida, igual que el listado de disponibles.

- Pedido asignado a otro repartidor: consulta y liberación devuelven 404.

- Pedido propio Preparando o Listo: se puede soltar y se registra el evento.

- Pedido propio Recibido, EnCamino, Entregado o Cancelado: no se puede soltar; se conserva toda la información.

- Pedido propio entregado: sigue visible en el historial de su repartidor. El PIN permanece oculto.

- Pedido cerrado sin asignar: no queda expuesto por conocer su ID.

Los datos necesarios para repartir siguen visibles en pedidos realmente disponibles. Reducir los datos personales mostrados antes de aceptar una entrega es una decisión adicional de privacidad, fuera de esta corrección.

Validación ejecutada

- Compilación .NET 10 sobre beta con -warnaserror: correcta, sin advertencias ni errores. La advertencia CS9113 observada en master ya estaba corregida en beta antes de integrar S01.

- Regresión API inicial S01: 62 comprobaciones correctas en una base demo nueva. La suite ampliada S01/S02 verificó 308 comprobaciones; la ampliación S01/S02/S03 verifica ahora 580.

- El rechazo de una liberación se compara con el pedido completo anterior: estado, repartidor, fechas y eventos permanecen iguales. Se comprueban también el acceso al historial, PIN oculto y límites entre usuarios/roles.

- Recorrido existente tests/e2e-capturas.cjs: completó registro, pago simulado, pedido, preparación, salida, rechazo de PIN incorrecto y entrega correcta, sin errores de JavaScript.

- Las pruebas se ejecutaron en localhost:5493 con datos desechables; no se enviaron solicitudes a producción.

Prueba de regresión, desde la raíz del repositorio, con una app local de desarrollo y base demo desechable:

```
Storage__DataRoot="$(mktemp -d)" dotnet run --project src/Diversify.Delivery --launch-profile http
# En otra terminal:
ALLOW_DEMO_MUTATIONS=1 BASE=http://localhost:5390 node tests/seguridad-repartidor.cjs
```

El script crea cliente y pedidos propios y restaura ajustes al terminar; no limpia la base. No usar con datos reales. La versión ampliada debe finalizar con 580 comprobaciones correctas. Las instrucciones completas están en tests/README.md.

Entrega y comprobaciones posteriores

Revisar la corrección integrada en beta. Antes de producción, corregir el pendiente de rutas a delivery.env y verificar el paquete. Tras publicar, repetir casos de acceso por estado y liberación con cuentas de prueba. Revisar posibles pedidos históricos sin atribución o con eventos de liberación posteriores al cierre; esta corrección no repara automáticamente datos alterados previamente ni demuestra que hubo abuso real.

La auditoría original se realizó sobre master, commit a286c42. Sus resultados originales se conservan en el entorno de revisión; no constituyen una nueva auditoría completa de beta. La validación pública de Caddy/Cloudflare, TLS, host, dependencias y pasarela real sigue fuera de lo probado. No se garantiza ausencia de otras vulnerabilidades.

Solución S02: asignación exclusiva de pedidos

Fecha: 9 de noviembre de 2026. Rama: beta. Corrección sobre bc24bbf.

Problema reproducido

Dos repartidores consultaban el mismo pedido sin dueño antes de que cualquiera guardara su asignación. Ambos recibían HTTP 200, aunque solo uno terminaba registrado en la base. La nueva regresión falló contra la versión anterior exactamente por ese resultado: [200, 200] en lugar de [200, 409].

Solución aplicada

Archivo: src/Diversify.Delivery/Modules/Orders.cs, POST /api/driver/orders/{id}/take.

La asignación es una única escritura condicional con ExecuteUpdateAsync: exige el ID solicitado, DriverId nulo y estado Preparando o Listo. La base devuelve cuántas filas cambió. Si cambia una fila, ese repartidor ganó; se añade un único evento y se confirma todo dentro de la misma transacción. No hay un intervalo entre consultar la disponibilidad y asignar el pedido. El evento y la asignación se confirman juntos o se revierten juntos.

Si no cambia ninguna fila, se comprueba el resultado persistido:

- Pedido inexistente: HTTP 404.

- Pedido de otro repartidor o estado no disponible: HTTP 409, con mensaje para actualizar la lista.

- Pedido que ya pertenece al mismo repartidor y sigue Preparando/Listo: HTTP 200 sin cambiar fechas ni añadir eventos. Permite reintentos ante pérdida de conexión sin repetir la toma.

Las respuestas 409 solo incluyen el error, sin datos personales del pedido. El PIN continúa oculto al repartidor. El cliente existente muestra el mensaje devuelto por la API. No se cambió el esquema ni se alteraron asignaciones históricas. El límite de pedidos se corrigió posteriormente en S03; ver `docs/SOLUCION-SEGURIDAD-S03.md`. El resto de operaciones concurrentes requiere su propia revisión.

Verificación

Prueba tests/seguridad-repartidor.cjs, ya incluida en CI:

- 12 rondas concurrentes entre dos cookies de repartidor, seis en Preparando y seis en Listo, alternando el orden.

- En todas las rondas, un ganador HTTP 200 y un conflicto HTTP 409.

- Verificación posterior por el master: dueño persistido igual al confirmado y exactamente un evento nuevo.

- El perdedor no puede consultar ni liberar el pedido.

- Dos reintentos paralelos del ganador mantienen idéntico el pedido completo: estado, fecha y eventos.

- Liberación válida entre rondas y rechazo de toma de un pedido cancelado sin modificarlo.

- Las regresiones previas de S01 se conservan. Resultado inicial S01/S02: 308 comprobaciones correctas; la suite ampliada con S03 verifica ahora 580 comprobaciones.

- Compilación con -warnaserror sin errores ni advertencias; JavaScript y ShellCheck correctos.

Las pruebas se hicieron con datos demo aislados en localhost. La prueba de concurrencia falla contra la versión anterior y pasa con la corrección. El recorrido completo de navegador terminó sin errores, desde el registro hasta la entrega con PIN correcto.

Reproducción local y entrega

Desde la raíz del repositorio, iniciar una instancia de desarrollo con base demo nueva:

```
Storage__DataRoot="$(mktemp -d)" dotnet run --project src/Diversify.Delivery --launch-profile http
# En otra terminal:
ALLOW_DEMO_MUTATIONS=1 BASE=http://localhost:5390 node tests/seguridad-repartidor.cjs
```

No usar con datos reales: la suite crea cliente y pedidos, ejecuta entregas y restaura los ajustes al finalizar. La corrección se entrega en beta. La ejecución local y el push no implican despliegue en producción ni que el CI remoto haya terminado. Tras desplegar, repetir la prueba con cuentas de prueba y revisar posibles asignaciones inconsistentes anteriores; el parche no repara automáticamente datos históricos.

Solución S03: máximo de tres pedidos sin atender

Fecha: 9 de noviembre de 2026. Rama: beta. Corrección sobre 8bfd1f3.

Problema reproducido

La aplicación contaba los pedidos Recibido antes de abrir la transacción que los insertaba. Varias solicitudes simultáneas podían ver el mismo cupo libre y crear más de tres pedidos para un cliente. La prueba nueva falló contra la versión anterior: ocho solicitudes simultáneas crearon ocho pedidos cuando solo debían aceptarse tres.

Solución aplicada

Archivo: src/Diversify.Delivery/Modules/Orders.cs, POST /api/orders/.

La comprobación del cupo se trasladó al interior de la transacción existente, antes de calcular el siguiente número e insertar el pedido. En Microsoft.Data.Sqlite esta transacción es serializable y no diferida: reserva la escritura al comenzar. El siguiente escritor solo puede contar después de que el anterior confirme o revierta su operación. No se emplea un candado en memoria que pudiera dejar de funcionar al ejecutar otra instancia de la aplicación.

Secuencia de la sección crítica:

1. Comenzar la transacción SQLite.

2. Contar los pedidos del cliente autenticado cuyo estado es Recibido.

3. Si hay tres o más, responder HTTP 400 con el mensaje de límite y revertir la transacción.

4. Si hay cupo, asignar el número correlativo e insertar el pedido, sus líneas y el evento inicial.

5. Confirmar la transacción y devolver HTTP 201.

Las validaciones de cuenta, dirección, carrito, productos, cantidades y pago permanecen antes de la sección crítica para mantener breve el bloqueo de escritura. No cambió el esquema de datos ni el contrato HTTP.

El límite se aplica por cliente a los pedidos sin atender en estado Recibido. Preparando y los demás estados no cuentan para ese cupo. Cancelar o empezar a preparar un pedido libera espacio. No se limita el total global de pedidos de la tienda ni el historial del cliente. La corrección no elimina ni cancela automáticamente pedidos excedentes que ya existieran antes del despliegue; si hay tres o más, rechaza nuevas creaciones hasta liberar cupo.

Pruebas y evidencia

La prueba tests/seguridad-repartidor.cjs, incluida en CI, conserva las regresiones S01 y S02 y añade:

- Ocho ráfagas de ocho solicitudes concurrentes (64 intentos): dos ráfagas con cada estado inicial de 0, 1, 2 o 3 pendientes.

- Creaciones aceptadas por ráfaga: exactamente 3, 2, 1 o 0, respectivamente; el resto responde HTTP 400 por el límite.

- Comprobación posterior de la base mediante la API: exactamente tres Recibido; los rechazos no crean filas.

- Números de pedido únicos después de cada ráfaga.

- Cancelación de un pendiente y creación de su reemplazo.

- Cambio de otro pendiente a Preparando y creación de su reemplazo.

- Un carrito inválido rechazado no consume cupo.

Resultado conjunto: 580 comprobaciones correctas con datos demo aislados. Compilación con -warnaserror, JavaScript y ShellCheck aprobados. El recorrido completo de navegador terminó sin errores, desde el registro hasta la entrega con PIN correcto. La prueba reproduce el exceso en la versión anterior y pasa con la corrección. No se hicieron pruebas de carga prolongadas ni se enviaron peticiones a producción.

Ejecución y entrega

Desde la raíz del repositorio, usar una instancia local con base demo nueva:

```
Storage__DataRoot="$(mktemp -d)" dotnet run --project src/Diversify.Delivery --launch-profile http
# En otra terminal:
ALLOW_DEMO_MUTATIONS=1 BASE=http://localhost:5390 node tests/seguridad-repartidor.cjs
```

Debe terminar con 580 comprobaciones correctas. La suite crea cliente y pedidos y restaura ajustes al finalizar; no usar con datos reales. No se desactivan las validaciones de negocio ni se cambia el límite del servidor.

La entrega se realiza en beta. El push no despliega producción. Tras desplegar, verificar el límite con cuentas de prueba y revisar si existen clientes con excedentes anteriores. La idempotencia de la creación ante reenvíos de red es una mejora adicional: este cambio garantiza el cupo, no identifica dos solicitudes como el mismo pedido. Los hallazgos S04–S08 siguen pendientes en notas/Pendientes.md.

Inventario completo de pendientes

Estado al cierre de esta entrega. S01, S02 y S03 están corregidos y validados localmente; el resto de casillas pendientes conserva su estado. Las decisiones de negocio requieren definición del responsable del producto.

Preguntas para definir (dueño del proyecto)

1. ~~Pasarela de pago~~ → **PixelPay** (definido 2026-11-09). Falta: credenciales del comercio (Key ID, llave secreta y URL del endpoint) y si cobra en HNL o USD.

2. **¿Un supermercado o varios?** ¿Delivery es solo para un súper (el de Diversify POS) o un mercado donde varios clientes de Diversify venden con sus propios repartidores? El modelo actual es un súper por instalación.

3. ~~La inscripción~~ → **pago único** (definido 2026-11-09). Por definir: ¿se reembolsa? ¿códigos promocionales?

4. **Facturación:** ¿el pedido debe crear la venta en Diversify POS (factura con CAI e inventario)? ¿Quién emite la factura y cuándo?

5. **Zonas y costo de envío:** ¿tarifa fija (hoy L 50) o por zona/colonia/distancia? ¿Hay zonas sin cobertura?

6. **Pagos de pedidos:** ¿efectivo, POS al recibir y transferencia son suficientes? ¿Pago en línea del pedido?

7. **Repartidores:** ¿empleados del súper o independientes? ¿Se les paga por entrega (comisión) y hay que llevar esa cuenta?

8. **Dominio:** ¿`delivery.diversify.hn` u otro?

9. **Notificaciones:** ¿WhatsApp automático (requiere WhatsApp Business API, con costo), SMS o correo?

10. **Logo propio** de "Diversify Delivery" o se queda la marca Diversify con la palabra DELIVERY.

Seguridad y operación (revisión del 9 de noviembre de 2026)

- [HECHO] S01: restringir consulta por ID a pedidos propios o disponibles (Preparando/Listo sin repartidor); impedir liberar pedidos Recibido, EnCamino, Entregado y Cancelado, conservando la atribución histórica. Validado con 62 comprobaciones de regresión. Solución en `docs/SOLUCION-SEGURIDAD-S01.md`.

- [HECHO] S02: asignación condicional atómica y evento en una transacción; el competidor recibe 409 y los reintentos del dueño no duplican eventos. Regresión S01/S02: 308 comprobaciones, incluidas 12 carreras entre dos sesiones. Detalle en `docs/SOLUCION-SEGURIDAD-S02.md`.

- [HECHO] S03: cupo, número e inserción en la misma transacción SQLite. Ocho ráfagas de ocho solicitudes con 0/1/2/3 pendientes conservan el máximo de tres. Suite S01/S02/S03: 580 comprobaciones correctas. Detalle en `docs/SOLUCION-SEGURIDAD-S03.md`.

- [PENDIENTE] S04: protección antiforgery en operaciones autenticadas con efectos; validar orígenes como defensa adicional.

- [PENDIENTE] S05: revocación de la sesión en servidor al cerrar sesión (no solo eliminación de la cookie).

- [PENDIENTE] S06: una transferencia pendiente por usuario, límites por cuenta/IP e idempotencia.

- [PENDIENTE] S07: rechazar métodos de pago no definidos y validar reglas de cada método.

- [PENDIENTE] S08: validar y normalizar imágenes, incluyendo las importadas de Diversify POS.

- [PENDIENTE] Corregir las rutas a `delivery.env` en systemd y actualización; probar instalación nueva.

- [PENDIENTE] Validar IP real detrás de Caddy/Cloudflare y evitar acceso directo a Kestrel; añadir límites por cuenta.

- [PENDIENTE] Generar PIN con criptografía y limitar intentos por pedido/cuenta.

- [HECHO] Advertencia CS9113 (`httpFactory` sin usar) ya corregida en beta antes de integrar S01; compilación con `-warnaserror` verificada sin advertencias.

- [PENDIENTE] Desplegar S01 y repetir la validación en el entorno publicado; revisar atribuciones de pedidos históricos que pudieran haber sido alteradas antes de la corrección, sin reconstruirlas automáticamente.

- [PENDIENTE] Retomar guardado de la configuración reutilizable del entorno: la plataforma no confirmó el borrador.

Técnico

- [HECHO] Cobro de la inscripción con PixelPay: Hosted Payment, orden firmada (SHA3-512), regreso y aviso (`_callback`) verificados, prueba automática. Revisado con la documentación oficial (MCP de PixelPay).

- [PENDIENTE] Probar PixelPay con el sandbox real desde una computadora (desde la nube lo bloquea Cloudflare) y luego en producción. Confirmar el formato del aviso (`_callback`) con un pago real.

- [PENDIENTE] Mapa dentro de la app para marcar la ubicación (hoy: GPS del celular + enlaces a Google Maps/Waze). Requiere decidir proveedor de mapas (y permitir su dominio en la CSP).

- [PENDIENTE] Notificaciones push / WhatsApp de cambios de estado.

- [HECHO] Recuperar contraseña por correo (enlace de 1 hora y un solo uso; prueba automática).

- [PENDIENTE] Elegir el servicio de correo (Brevo, Gmail…) y configurarlo en el servidor (`Email__*`).

- [PENDIENTE] Enviar el pedido entregado a Diversify POS como venta (ver pregunta 4).

- [PENDIENTE] EF Core Migrations (hoy `EnsureCreated` + `SchemaUpgrades`, igual que Diversify POS).

- [PENDIENTE] Términos y condiciones y aviso de privacidad (texto legal).

- [PENDIENTE] PWA instalable (manifest + service worker) para clientes y repartidores.

- [PENDIENTE] Probar la instalación en el Proxmox real junto con Diversify POS.

Hecho

- [HECHO] Tienda en línea: catálogo público, búsqueda, ofertas, carrito, direcciones con GPS, pedido, seguimiento.

- [HECHO] Registro con inscripción de US$5 (tarjeta simulada / transferencia con aprobación).

- [HECHO] Panel master: tablero, pedidos kanban, ajuste de productos, asignación, productos y precios (rápido, masivo, historial), clientes e inscripciones, repartidores, ajustes.

- [HECHO] App del repartidor: disponibles, mis entregas, navegación, llamar/WhatsApp, entrega con código.

- [HECHO] Sincronización con Diversify POS (probada en local).

- [HECHO] Scripts de despliegue para un contenedor LXC aparte.

- [HECHO] Prueba de punta a punta con Playwright y capturas.

Detalle de los hallazgos aún pendientes

Evidencia de la auditoría local anterior a esta corrección. Los resultados siguientes no se presentan como reparados. La severidad es cualitativa y las condiciones de explotación se indican en cada caso. Actualización al integrar beta: CS9113 ya estaba corregido; la compilación estricta de esta entrega pasó sin advertencias. PixelPay y recuperación de contraseña ya están implementados según el inventario actualizado; sus pruebas con servicios reales siguen pendientes.

S04 · Media, condicionada al origen atacante · CSRF en acciones con cookie

Ubicación: Program.cs:34–46 y endpoints POST sin validación antiforgery, especialmente Orders.cs:402.

SameSite=Lax separa sitios, no orígenes. Una página controlada por un atacante en otro origen del mismo sitio puede emitir un POST simple con credentials:include. Las acciones sin cuerpo no requieren JSON ni preflight. No se valida token antiforgery ni Origin.

Prueba real en Chromium: sesión de repartidor en http://127.0.0.1:5491; página atacante en http://127.0.0.1:5492; POST simple a /api/driver/orders/4/take. CORS bloqueó leer la respuesta, pero la acción sí asignó el pedido al repartidor 2. La sesión se había autenticado antes de abrir el origen atacante.

Condición: en producción requiere que el atacante controle un origen del mismo sitio (por ejemplo, un subdominio hermano bajo el mismo dominio registrable) o una posición equivalente. No se demostró CSRF autenticado desde un sitio externo arbitrario. La CSP del destino y CORS no evitan esta escritura.

Corrección: antiforgery en todas las operaciones con efectos, integrado con el cliente JavaScript; validar Origin/Referer y Fetch Metadata como defensa adicional. No confiar solo en SameSite ni en exigir JSON en algunos endpoints.

S05 · Media · Cerrar sesión no revoca una copia de la cookie

Ubicación: Modules/AuthEndpoints.cs:60–64; Core/Security.cs:45–87; Program.cs:39–40.

Logout solo elimina la cookie del navegador. Una copia previa de la cookie siguió obteniendo HTTP 200 en /api/driver/orders después de que logout respondiera 204. La cookie está configurada con expiración de 30 días y renovación deslizante.

Condición: el atacante debe haber obtenido una cookie válida; no se encontró una vía de robo de cookie en estas pruebas. El fallo es la ausencia de revocación después de logout, no un bypass de autenticación sin credenciales. No se midió la supervivencia durante 30 días.

Corrección: identificador de sesión y estado de revocación en servidor (ITicketStore o sesiones persistidas); revocar esa sesión al salir y ofrecer cierre de todas las sesiones. El stamp actual depende de propiedades del usuario, no de una sesión individual.

Controles positivos: cambiar contraseña invalidó la cookie anterior (401); suspender el usuario invalidó su sesión (401).

S06 · Media · Registros de transferencia sin límite por cuenta

Ubicación: Modules/RegistrationEndpoints.cs:87–108; no se aplica RequireRateLimiting a esa operación.

Una cuenta recién registrada pudo enviar 12 referencias diferentes en secuencia: las 12 devolvieron HTTP 200 y crearon solicitudes de revisión. No se exige resolver una solicitud pendiente antes de crear otra ni hay tope por cuenta. Esto puede llenar la cola del master y generar escrituras y registros innecesarios.

La prueba fue acotada; no se ejecutó una prueba de carga ni se demostró caída del servidor. Tampoco se movió dinero real.

Corrección: una única solicitud pendiente por usuario, cuota por cuenta e IP, idempotencia y restricciones de unicidad/estado en base de datos. Considerar verificación de correo y tratamiento de abuso.

S07 · Baja · Método de pago fuera del enum aceptado

Ubicación: Program.cs:18–19 y Modules/Orders.cs:189–207.

Enviar paymentMethod:999 creó un pedido con HTTP 201 y paymentMethod:"999". JsonStringEnumConverter permite enteros y el endpoint no verifica Enum.IsDefined. Esto persiste estados no soportados y evita la rama de validación específica de Efectivo.

No se demostró obtención de productos gratis: el mismo intento de enviar total:0 y price:0 fue ignorado y el servidor calculó L 550, con subtotal esperado L 500 y envío L 50.

Corrección: rechazar valores no definidos, desactivar enteros en el conversor si no son parte del contrato y validar las reglas de cada método de pago en servidor.

S08 · Baja · Imagen inválida aceptada y error público 500

Ubicación: Core/Formats.cs:25–31; Modules/CatalogEndpoints.cs:60–67.

El master pudo crear un producto con imageData:"!": HTTP 201. La lectura pública de su imagen respondió HTTP 500 porque se decodifica sin validar previamente Base64 ni contenido real.

Requiere permisos de master para la carga directa; por eso no se califica como caída pública de toda la aplicación ni como ejecución de código. La importación desde POS también copia ImageData sin pasar por Text.Image (DiversifySync.cs:147), lo que amplía el problema a datos de integración y merece validar MIME y contenido antes de servirlos.

Corrección: decodificar al importar/subir, limitar bytes y dimensiones, comprobar formato real y normalizar a una imagen permitida. Aplicar la misma validación a la sincronización. Una imagen inválida debe producir 400 al cargarse y no un 500 en el catálogo.

Riesgos de despliegue y observaciones separadas

- **Configuración de systemd inconsistente, confirmada por revisión:** deploy/install.sh crea /etc/diversify-delivery/delivery.env, pero deploy/systemd/diversify-delivery.service:13 y deploy/update.sh:19 usan /etc/diversify-delivery-delivery/delivery.env. Con los archivos tal como están, una instalación nueva puede no arrancar y una actualización no leer el puerto correcto. No se ejecutó una instalación de producción; corregir las rutas y probar el paquete en un contenedor desechable.

- **Límite por IP dependiente del proxy:** tras alcanzar 429 en localhost, cambiar X-Forwarded-For devolvió 401 y permitió otro intento. El peer local es un proxy de confianza de ASP.NET. Esto confirma el comportamiento en acceso directo local, no un bypass remoto en Caddy. Mantener Kestrel en loopback, impedir acceso directo y verificar que Caddy/Cloudflare reescriben y no aceptan IP arbitraria del visitante. Añadir límite por cuenta; el PIN de entrega comparte la política por IP.

- **PIN:** se genera con Random.Shared.Next(1000,10000). Usar RandomNumberGenerator.GetInt32 y un presupuesto de intentos por pedido/cuenta, además del límite por IP. No se intentó fuerza bruta completa ni se predijo un PIN.

- **CI:** la compilación limpia con -warnaserror falla por CS9113, parámetro httpFactory sin usar en Modules/DiversifySync.cs:25. Defecto de calidad/operación, no vulnerabilidad de explotación demostrada.